

Franck Pachot, dbi services

Parallel Distribution and 12c Adaptive Plans

In the previous newsletter we have seen how 12c can defer the choice of the join method to the first execution. We considered only serial execution plans. But besides join method, the cardinality estimation is a key decision for parallel distribution when joining in parallel query. Ever seen a parallel query consuming huge tempfile space because a large table is broadcasted to lot of parallel processes? This is the point addressed by Adaptive Parallel Distribution.

Once again, that new feature is a good occasion to look at the different distribution methods.

Parallel Query Distribution

I'll do the same query as in previous newsletter, joining EMP with DEPT, but now I choose to set a parallel degree 4 to the EMP table. If I do the same hash join as before, DEPT being the built table, I will have:

- Four consumer processes that will do the Hash Join.
 - One process (the coordinator) reading DEPT which is not in parallel – and sending rows to one of the consumer processes, depending on the hash value calculated from on the join column values.
 - Each of the four consumers receives their part of the DEPT rows and hash them to create their built table.
 - Four producer processes, each reading specific granules of EMP, send each row to one of the four consumer.
 - Each of the four consumers receives their part of EMP rows and matches them to their probe table.
 - Each of them sends their result to the coordinator.
- Because the work was divided with a hash function on the join column, the final result of the join is just the concatenation of each consumer result.

Here is the execution plan for that join:

EXPLAINED SQL STATEMENT:										

select * from DEPT join EMP using(deptno)										

	Id	Operation	Name	Starts	TQ	IN-OUT	PQ Distrib	A-Rows	Buffers	OMem

	0	SELECT STATEMENT		1				14	10	
	1	PX COORDINATOR		1				14	10	
	2	PX SEND QC (RANDOM)	:TQ10002	0	Q1,02	P->S	QC (RAND)	0	0	
*	3	HASH JOIN BUFFERED		4	Q1,02	PCWP		14	0	1542K
	4	BUFFER SORT		4	Q1,02	PCWC		4	0	2048
	5	PX RECEIVE		4	Q1,02	PCWP		4	0	
	6	PX SEND HASH	:TQ10000	0		S->P	HASH	0	0	
	7	TABLE ACCESS FULL	DEPT	1				4	7	
	8	PX RECEIVE		3	Q1,02	PCWP		14	0	
	9	PX SEND HASH	:TQ10001	0	Q1,01	P->P	HASH	0	0	
	10	PX BLOCK ITERATOR		4	Q1,01	PCWC		14	15	
*	11	TABLE ACCESS FULL	EMP	5	Q1,01	PCWP		14	15	

Execution Plan 1: PX hash distribution

The Q1,01 is the producer set that reads EMP, the Q1,02 is the consumer set that does the join. The 'PQ Distrib' column shows the HASH distribution for both the outer rowsource DEPT and the inner table EMP. The hint for that is PQ_DISTRIBUTE(DEPT HASH HASH) to be added to the leading(EMP DEPT) use_hash(DEPT) swap_join_inputs(DEPT) that defines the join order and method.

This is efficient when both tables are big. But with a DOP of 4 we have $1+2*4=8$ processes and a lot of messaging among them.

When one table is not so big, then we can avoid a whole set of parallel processes. We can broadcast the small table (DEPT) to the 4 parallel processes doing the join. In that case, the same set of processes is able to read EMP and do the join.

Here is the execution plan:

EXPLAINED SQL STATEMENT:

```
select /** leading(EMP DEPT) use_hash(DEPT) swap_join_inputs(DEPT) pq_distribute(DEPT NONE BROADCAST) */ * from DEPT join EMP using(deptno)
```

Id	Operation	Name	Starts	TQ	IN-OUT	PQ Distrib	A-Rows	Buffers	OMem
0	SELECT STATEMENT		1				14	10	
1	PX COORDINATOR		1				14	10	
2	PX SEND QC (RANDOM)	:TQ10001	0	Q1,01	P->S	QC (RAND)	0	0	
* 3	HASH JOIN		4	Q1,01	PCWP		14	15	1321K
4	BUFFER SORT		4	Q1,01	PCWC		16	0	2048
5	PX RECEIVE		4	Q1,01	PCWP		16	0	
6	PX SEND BROADCAST	:TQ10000	0		S->P	BROADCAST	0	0	
7	TABLE ACCESS FULL	DEPT	1				4	7	
8	PX BLOCK ITERATOR		4	Q1,01	PCWC		14	15	
* 9	TABLE ACCESS FULL	EMP	5	Q1,01	PCWP		14	15	

Execution Plan 2: PX broadcast from serial

The coordinator reads DEPT and broadcasts all rows to each parallel server process (Q1,01). Those processes build the hash table for DEPT and then read their granules of EMP.

With the PQ_DISTRIBUTE we can choose how to distribute a table to the consumer that will process the rows. The syntax is PQ_DISTRIBUTE(inner_table outer_distribution inner_distribution). For HASH we must use the same hash function, so we will see PQ_DISTRIBUTE(DEPT HASH HASH) for producers sending to consumer according to the hash function.

We can choose to broadcast the inner table with PQ_DISTRIBUTE(DEPT NONE BROADCAST) or the outer rowsource PQ_DISTRIBUTE(DEPT BROADCAST NONE). The broadcasted table will be received in a whole by each consumer, so it can take a lot of memory, when it is buffered by the join operation and when the DOP is high.

When the tables are partitioned, the consumers can divide their job by partitions instead of granules, and we can distribute rows that match each consumer partition. For example, if EMP is partitioned on DEPTNO, then PQ_DISTRIBUTE(DEPT NONE PARTITION) will distribute the DEPT rows to the right consumer process according to DEPTNO value. The opposite PQ_DISTRIBUTE (DEPT PARTITION NONE) would be done, if DEPT were partitioned on DEPTNO.

And if both EMP and DEPT are partitioned on DEPTNO, then there is nothing to distribute: PQ_DISTRIBUTE(DEPT NONE NONE) because each parallel process is able to read both EMP and DEPT partition and do the Hash Join. This is known as partition-wise join and is very efficient when the number of partition is equal to the DOP, or a large multiple.

12c Small Table Replicate

If we take the example above where DEPT was broadcasted, but setting a parallel degree on DEPT as well, we have the following execution plan:

Id	Operation	Name	Starts	TQ	IN-OUT	PQ Distrib	A-Rows	Buffers	OMem
0	SELECT STATEMENT		1				14	6	
1	PX COORDINATOR		1				14	6	
2	PX SEND QC (RANDOM)	:TQ10001	0	Q1,01	P->S	QC (RAND)	0	0	
* 3	HASH JOIN		4	Q1,01	PCWP		14	15	1321K
4	PX RECEIVE		4	Q1,01	PCWP		16	0	
5	PX SEND BROADCAST	:TQ10000	0	Q1,00	P->P	BROADCAST	0	0	
6	PX BLOCK ITERATOR		4	Q1,00	PCWC		4	15	
* 7	TABLE ACCESS FULL	DEPT	5	Q1,00	PCWP		4	15	
8	PX BLOCK ITERATOR		4	Q1,01	PCWC		14	15	
* 9	TABLE ACCESS FULL	EMP	5	Q1,01	PCWP		14	15	

Execution Plan 3: PX broadcast from parallel

Here we have a set of producers (Q1,00) that will broadcast to all consumers (Q1,01). That was the behavior in 11g.

In 12c a step further than broadcasting can be done by replicating the reading of DEPT in all consumers instead of broadcasting.

Id	Operation	Name	Starts	TQ	IN-OUT	PQ Distrib	A-Rows	Buffers	OMem
0	SELECT STATEMENT		1				14	3	
1	PX COORDINATOR		1				14	3	
2	PX SEND QC (RANDOM)	:TQ10000	0	Q1,00	P->S	QC (RAND)	0	0	
* 3	HASH JOIN		4	Q1,00	PCWP		14	43	1321K
4	TABLE ACCESS FULL	DEPT	4	Q1,00	PCWP		16	28	
5	PX BLOCK ITERATOR		4	Q1,00	PCWC		14	15	
* 6	TABLE ACCESS FULL	EMP	5	Q1,00	PCWP		14	15	

Execution Plan 4: PQ replicate

That optimization requires more I/O (but it concerns only small tables anyway – in can be cached when using In-Memory parallel execution) but saves processes, memory and messaging. The hint is PQ_DISTRIBUTE(DEPT NONE BROADCAST) PQ_REPLICATE(DEPT)

12c Adaptive Parallel Distribution

12c comes with Adaptive Plans. We have seen in the previous newsletter the Adaptive Join when it is difficult to estimate the cardinality and to choose between Nested Loop and Hash Join. It is the same concern here when choosing between broadcast and hash distribution: Adaptive Parallel Distribution.

The previous HASH HASH parallel plans were done in 11g. Here is the same in 12c:

EXPLAINED SQL STATEMENT:									
select * from DEPT join EMP using(deptno)									
Id	Operation	Name	Starts	TQ	IN-OUT	PQ Distrib	A-Rows	Buffers	OMem
0	SELECT STATEMENT		1				14	10	
1	PX COORDINATOR		1				14	10	
2	PX SEND QC (RANDOM)	:TQ10002	0	Q1,02	P->S	QC (RAND)	0	0	
* 3	HASH JOIN BUFFERED		4	Q1,02	PCWP		14	0	1542K
4	BUFFER SORT		4	Q1,02	PCWC		16	0	2048
5	PX RECEIVE		4	Q1,02	PCWP		16	0	
6	PX SEND HYBRID HASH	:TQ10000	0		S->P	HYBRID HASH	0	0	
7	STATISTICS COLLECTOR		1				4	7	
8	TABLE ACCESS FULL	DEPT	1				4	7	
9	PX RECEIVE		4	Q1,02	PCWP		14	0	
10	PX SEND HYBRID HASH	:TQ10001	0	Q1,01	P->P	HYBRID HASH	0	0	
11	PX BLOCK ITERATOR		4	Q1,01	PCWC		14	15	
* 12	TABLE ACCESS FULL	EMP	5	Q1,01	PCWP		14	15	

Execution Plan 5: Adaptive Parallel Distribution

The distribution is HYBRID HASH and there is a STATISTICS COLLECTOR before sending to parallel server consumers. Oracle will count the rows coming from DEPT and will choose to BROADCAST or HASH depending on the number of rows.

It is easy to check what has been chosen here, knowing that the DOP was 4. I have 4 rows coming from DEPT ('A-rows' on DEPT TABLE ACCESS FULL) and 16 were received by the consumer ('A-Rows' on PX RECEIVE): this is broadcast ($4 \times 4 = 16$).

Parallel Query Distribution from SQL Monitoring

When we have the Tuning Pack, it is easier to get execution statistics from SQL Monitoring. Here are the same execution plans as above, but gathered with SQL Monitoring reports.

The coordinator in green does everything that is done in serial. The producers are in blue, the consumers are in red.

Here is the Hash distribution where DEPT read in serial and EMP read in parallel are both distributed to the right consumer that does the join:

Operation	Name	Line ID	Estimated Rows	Cost	Executions	Actual Rows	Memory (MB)	Other
SELECT STATEMENT		0		9	9	14		
PX COORDINATOR		1		9	9	14		
PX SEND QC (RANDOM)	/TQ00002	2	14	9	9	14		
HASH JOIN BROADCAST		3	14	9	9	14		3MB
TABLE ACCESS FULL	DEPT	4	4	3	4	4		
PX RECEIVE		5	14	2	4	14		
PX SEND HASH	/TQ00003	6	14	2	4	14		
PX BLOCK ITERATOR		7	14	2	4	14		
TABLE ACCESS FULL	EMP	8	14	2	4	14		

SQL Monitor 1: PX hash distribution

Here is the broadcast from DEPT serial read:

Operation	Name	Line ID	Estimated Rows	Cost	Executions	Actual Rows	Memory (MB)	Other
SELECT STATEMENT		0		9	9	14		
PX COORDINATOR		1		9	9	14		
PX SEND QC (RANDOM)	/TQ00002	2	14	9	9	14		
HASH JOIN		3	14	9	9	14		3MB
TABLE ACCESS FULL	DEPT	4	4	3	4	4		
PX RECEIVE		5	14	2	4	14		
PX SEND BROADCAST	/TQ00003	6	14	2	4	14		
PX BLOCK ITERATOR		7	14	2	4	14		
TABLE ACCESS FULL	EMP	8	14	2	4	14		

SQL Monitor 2: PX broadcast from serial

And the broadcast from DEPT parallel read (two sets of parallel servers):

Operation	Name	Line ID	Estimated Rows	Cost	Executions	Actual Rows	Memory (MB)	Other
SELECT STATEMENT		0		9	9	14		
PX COORDINATOR		1		9	9	14		
PX SEND QC (RANDOM)	/TQ00002	2	14	9	9	14		
HASH JOIN		3	14	9	9	14		3MB
TABLE ACCESS FULL	DEPT	4	4	3	4	4		
PX RECEIVE		5	14	2	4	14		
PX SEND BROADCAST	/TQ00003	6	14	2	4	14		
PX BLOCK ITERATOR		7	14	2	4	14		
TABLE ACCESS FULL	EMP	8	14	2	4	14		

SQL Monitor 3: PX broadcast from parallel

Then here is the 12c Small Table Replicate allowing to read DEPT from the same set of parallel processes that is doing the join:

Operation	Name	Line ID	Estimated Rows	Cost	Executions	Actual Rows	Memory (MB)	Other
SELECT STATEMENT		0		9	9	14		
PX COORDINATOR		1		9	9	14		
PX SEND QC (RANDOM)	/TQ00002	2	14	9	9	14		
HASH JOIN		3	14	9	9	14		3MB
TABLE ACCESS FULL	DEPT	4	4	3	4	4		
PX BLOCK ITERATOR		5	14	2	4	14		
TABLE ACCESS FULL	EMP	6	14	2	4	14		

SQL Monitor 4: PQ replicate

And in 12c, the choice between HASH and BROADCAST being done at runtime, and called HYBRID HASH:

Operation	Name	Line ID	Estimated Rows	Cost	Executions	Actual Rows	Memory (MB)	Other
SELECT STATEMENT		0		9	9	14		
PX COORDINATOR		1		9	9	14		
PX SEND QC (RANDOM)	/TQ00002	2	14	9	9	14		
HASH JOIN BROADCAST		3	14	9	9	14		3MB
TABLE ACCESS FULL	DEPT	4	4	3	4	4		
PX RECEIVE		5	14	2	4	14		
PX SEND HYBRID HASH	/TQ00003	6	14	2	4	14		
STATISTICS COLLECTOR		7						
TABLE ACCESS FULL	DEPT	8	4	3	4	4		
PX RECEIVE		9	14	2	4	14		
PX SEND HYBRID HASH	/TQ00004	10	14	2	4	14		
PX BLOCK ITERATOR		11	14	2	4	14		
TABLE ACCESS FULL	EMP	12	14	2	4	14		

SQL Monitor 5: Adaptive Parallel Distribution

Conclusion

Long before MapReduce became a buzzword, Oracle was able to distribute the processing of SQL queries to several parallel processes (and to several nodes when in RAC). Reading a table in parallel is easy: Each process reads a separate chunk. But when we need to join tables, then the rows have to be distributed from a set of producers (which full scan their chunks) to a set of consumers (which will do the join). Small row sets do not need to be processed in parallel and can be broadcasted to each consumer. But large rowset will be distributed to the right process only. The choice depends on the size and then the Cost Based Optimizer estimation of cardinality is a key point.

As we have seen for join methods, Oracle 12c can defer that choice to the first execution. This is Adaptive Parallel Distribution. ■

Contact

dbi services

Franck Pachot

E-Mail:

franck.pachot@dbi-services.com